

Methodological Support Report

SDR Spectrum Monitoring Sensor Node

Repository-grounded architectural and design analysis

Artifact type:	Technical methodological report
System class:	Embedded cyber-physical SDR sensing node
Primary implementation languages:	C and Python
Compilation baseline:	CMake + Python virtual environment
Document date:	May 22, 2026

This document was prepared from direct inspection of the repository source code, build scripts, runtime entry points, documentation stubs, and verification assets. The report therefore describes the implemented system as it exists in the inspected codebase, rather than an idealized target architecture.

Abstract

This report provides methodological support for the SDR Spectrum Monitoring Sensor node implemented in the repository. The analysis frames the node as a layered measurement system that combines deterministic radio-frequency acquisition in C with supervisory orchestration in Python. The report documents the architectural foundation, allocation of responsibilities, inter-process integration mechanisms, resilience patterns, deployment model, and current verification evidence. Each analytical section explicitly states its scope, technical basis, and module-level realization so that the text can serve both as formal design documentation and as a starting point for future maintenance, auditing, and extension activities.

Contents

Abstract	i
1 Purpose and Methodological Basis	2
1.1 Document Objective	2
1.2 Analytical Evidence Base	2
2 System Foundation	4
2.1 Operational Problem Definition	4
2.2 Architectural Partitioning Principle	4
3 Architectural Decomposition	6
3.1 RF Acquisition and DSP Subsystem	6
3.2 Configuration Integrity for RF Requests	6
3.3 Calibration and Frequency Error Compensation	7
3.4 GPS and LTE Subsystem	7
3.5 Hardware GPIO Abstraction	8
3.6 Orchestration and State Management	8
3.7 Campaign Scheduling and Deferred Execution	8
3.8 Inter-process Communication and Shared State	9
3.9 Data Integrity, Retry, and Offline Survivability	9
3.10 Observability and Operational Telemetry	9
3.11 Audio Streaming and Remote Listening	10
4 Design Rationale and Quality Attributes	11
4.1 Determinism and Timing Discipline	11
4.2 Fault Containment and Recoverability	11
4.3 Maintainability and Extension Potential	12
4.4 Resource Stewardship in an Embedded Context	12
5 Build, Deployment, and Documentation Method	13
5.1 Compilation Strategy	13

5.2	Deployment and Service Registration	13
5.3	Documentation Pipeline	14
6	Verification Status and Methodological Gaps	15
6.1	Current Verification Evidence	15
6.2	Limitations Identified During Inspection	15
6.3	Recommended Verification Priorities	16
7	Traceability Matrix	17
7.1	Requirement-to-Module Allocation	17
8	Conclusion	19
8.1	Synthesis	19
A	Key Source Inventory	20

Chapter 1

Purpose and Methodological Basis

1.1 Document Objective

Scope. The purpose of this report is to formalize the engineering rationale behind the sensor node, with emphasis on architectural decomposition, module responsibilities, design trade-offs, and evidence of implementation. The report is not limited to user-facing behavior; it addresses the full technical chain from hardware control and signal processing to orchestration, telemetry, resilience, and deployment.

Technical approach. The document follows a systems-engineering method based on three principles:

1. requirement-oriented decomposition of the sensing problem into acquisition, processing, coordination, telemetry, and recoverability concerns;
2. architectural allocation of each concern to the most appropriate runtime domain according to timing sensitivity, hardware coupling, and operational risk;
3. traceability from design intent to concrete source modules and scripts.

This method is appropriate for mixed embedded systems because it avoids both extremes of over-centralization and undocumented ad hoc growth.

Code realization. The repository already exhibits a strong separation of concerns that supports this method: real-time RF execution is concentrated in `rf/rf.c` and `rf/libs/*`, telecommunication and positioning are handled in `gps-lte/gps-lte.c` and `gps-lte/libs/*`, orchestration is centered in `orchestrator.py` and `functions.py`, and cross-cutting infrastructure is concentrated in `cfg.py`, `utils/io/_util.py`, `utils/request/_util.py`, and `utils/status/_util.py`.

1.2 Analytical Evidence Base

Scope. This section defines the evidentiary basis used to formulate the report so that the document remains auditable.

Technical approach. The analysis uses repository-internal evidence only. Source inspection focused on runtime entry points, public interfaces, build scripts, deployment scripts, documentation generators, and available tests or operational validation scripts. This avoids introducing

unsupported assumptions about hardware behavior or service contracts that are not visible in the repository.

Code realization. The principal evidence sources are the top-level execution modules `orchestrator.py`, `campaign\_runner.py`, `status.py`, `retry\_queue.py`, `server\_webrtc.py`, `rf/rf.c`, and `gps-lte/gps-lte.c`; the build scripts `CMakeLists.txt`, `build.sh`, and `install.sh`; the documentation scaffolding `document.sh`, `docs/python\_modules.rst`, and `docs/c\_modules.rst`; and the validation assets under `test/` and `examples/`.

Chapter 2

System Foundation

2.1 Operational Problem Definition

Scope. The node is designed to function as an embedded spectral-monitoring endpoint that can receive RF acquisition commands, calculate spectral products, optionally demodulate audio, collect GPS coordinates, maintain network connectivity, and publish telemetry to remote services.

Technical approach. From a theoretical perspective, the system belongs to the class of distributed instrumentation nodes. Such systems must reconcile three competing concerns: deterministic data-path execution, flexible supervisory logic, and survivability under intermittent connectivity or partial hardware failure. The adopted architecture therefore separates hard real-time or near-real-time operations from policy-driven orchestration.

Code realization. The C subsystem implements the time-sensitive path: HackRF control, IQ ingestion, ring-buffer management, PSD estimation, demodulation, calibration, GPIO selection, and ZMQ reply generation. The Python subsystem implements policy-heavy tasks: state transitions, polling the API, campaign scheduling, retry storage, status reporting, and WebRTC process lifecycle management. Shared state is maintained through `/dev/shm/persistent.json`, exposed in Python through `ShmStore` and in the GPS-LTE C service through dedicated SHM helper functions.

2.2 Architectural Partitioning Principle

Scope. This section explains why the repository is intentionally partitioned between C and Python instead of converging on a single implementation language.

Technical approach. The partitioning strategy follows a performance-risk alignment principle. C is used where the software directly faces high-rate hardware streams, strict memory reuse requirements, callback contexts, and DSP kernels. Python is used where the dominant concerns are asynchronous orchestration, serialization, network communication, scheduling, and operator-facing maintainability. This is not merely a convenience choice; it is a methodological decision that minimizes timing jitter in the data plane while preserving high development velocity in the control plane.

Code realization. The decision is visible in the repository layout and in the runtime call graph. `rf/rf.c`, `rf/libs/psd.c`, `rf/libs/chan\_filter.c`, and `rf/libs/sdr\_HAL.c` keep the RF path native and memory-conscious. By contrast, `orchestrator.py`, `campaign\_runner.py`, and `retry\_queue.py` rely on Python’s dataclasses, HTTP libraries, asynchronous flow control, and OS process management to implement higher-level behavior.

Chapter 3

Architectural Decomposition

3.1 RF Acquisition and DSP Subsystem

Scope. This subsystem covers radio hardware control, IQ capture, spectrum estimation, demodulation, audio streaming preparation, calibration, and response serialization. It is the core measurement engine of the node.

Technical approach. The subsystem follows a staged streaming architecture. A high-frequency hardware callback performs minimal work and forwards raw IQ samples into ring buffers. Downstream processing then operates outside the callback context, enabling PSD estimation, demodulation, filtering, and JSON reply construction without risking upstream sample loss. This is a standard embedded DSP pattern for decoupling acquisition from computation.

Code realization. The principal implementation is `rf/rf.c`. The function `rx\_callback` writes data into the primary ring buffer and conditionally mirrors IQ samples into an audio buffer. Configuration requests are parsed into `DesiredCfg\_t` through `parse\_config\_rf` in `rf/libs/parser.c`, mapped to hardware and PSD settings via `find\_params\_psd` in `rf/libs/psd.c`, and applied to the HackRF through `hackrf\_apply\_cfg` in `rf/libs/sdr\_HAL.c`. The RF engine stores reusable workspaces in dynamic buffers to avoid repeated allocation costs during repeated acquisitions. Results are serialized by `publish\_results`, which emits PSD arrays and mode-specific metrics such as AM depth or FM excursion.

3.2 Configuration Integrity for RF Requests

Scope. This section addresses how the node prevents malformed or physically inconsistent runtime requests from propagating directly into the RF pipeline.

Technical approach. The design applies two validation layers. The first layer is a typed Python-side request contract that rejects obviously invalid user or server input. The second layer is a C-side parser that enforces defaults and clamps filter limits to the Nyquist region derived from the requested center frequency and sample rate. The dual-layer strategy is methodologically sound because the Python layer protects normal control flow while the C layer remains authoritative for the actual measurement engine.

Code realization. On the Python side, `ServerRealtimeConfig` in `utils/request\_util.py` validates frequency range, sample-rate bounds, antenna port values, PSD method names, demodulation mode, and non-negative cooldown intervals. On the native side, `rf/libs/parser.c` sets factory-safe defaults, parses the JSON request, and clamps the optional `filter` object so that its start and end frequencies remain inside the available capture bandwidth. This explicit clamping is one of the most important correctness barriers in the RF path.

3.3 Calibration and Frequency Error Compensation

Scope. The calibration path exists to estimate frequency error and propagate the resulting parts-per-million correction through subsequent acquisitions.

Technical approach. Calibration is treated as a closed-loop estimation problem. The system captures a spectrum, identifies candidate peaks, evaluates FM pilot-tone evidence, and derives a correction factor that can be persisted and reused. Methodologically, this is preferable to hardcoded oscillator correction because it adapts to unit-specific hardware drift and environmental variation.

Code realization. The native calibration routine is implemented in `calibrate\_hackrf` inside `rf/rf.c`. Successful calibration concludes with persistence of `ppm\_error` into the shared-memory store. During normal acquisition, `fetch\_realtime\_config` in `orchestrator.py` retrieves the current correction from `ShmStore`, and `hackrf\_apply\_cfg` in `rf/libs/sdr\_HAL.c` computes a corrected tuning frequency using the standard linear PPM formula. This makes the correction operational rather than merely diagnostic.

3.4 GPS and LTE Subsystem

Scope. This subsystem manages modem control, PPP connectivity, GPS acquisition, coordinate normalization, and publication of mobility information to the external API and local shared state.

Technical approach. The design treats connectivity and geolocation as a separate hardware service rather than an auxiliary function inside the RF process. This separation is methodologically justified because LTE modem control and GPS parsing involve distinct peripherals, blocking I/O patterns, and failure modes that should not perturb the RF sampling pipeline.

Code realization. The executable entry point is `gps-lte/gps-lte.c`. It contains strict parsing for numeric coordinates, NMEA-to-decimal conversion helpers, haversine-based displacement checks, and logic for PPP recovery through `connection\_LTE`. The subordinate libraries `gps-lte/libs/bacn\_GPS.c`, `gps-lte/libs/bacn\_LTE.c`, and `gps-lte/libs/Utils.c` encapsulate GPS UART handling, AT-command transport, HTTP posting, and shared-memory persistence. The module also writes `last\_lat`, `last\_lng`, and `changed\_gps` so that location state becomes available to the rest of the node without direct process coupling.

3.5 Hardware GPIO Abstraction

Scope. GPIO support provides board-specific control of modem power, reset behavior, status lines, and antenna routing.

Technical approach. The design uses a minimal hardware abstraction layer around `libgpiod`. This is a sound methodological compromise: the code remains close enough to the platform to preserve exact pin semantics, but it avoids scattering raw GPIO requests across unrelated application modules.

Code realization. The abstraction is implemented in `common/bacn\_gpio.c` and `common/bacn\_gpio.h`. These files centralize pin offsets, line request helpers, modem power control routines, reset sequencing, status inspection, and antenna selection functions. The build system can exclude this dependency through `BUILD\_STANDALONE`, which preserves local development on non-target hosts.

3.6 Orchestration and State Management

Scope. The orchestration layer determines what the node should do at any moment: remain idle, serve realtime acquisitions, execute campaigns, or perform calibration.

Technical approach. The repository adopts a supervisory finite-state approach. Mutually exclusive high-level modes are represented explicitly, which prevents accidental overlap between incompatible hardware operations such as calibration and campaign-driven acquisition. This approach is more robust than an implicit state model spread across timers and boolean flags.

Code realization. The state machine is centered on `SysState` and `GlobalSys` in `functions.py`. `orchestrator.py` implements the control loop, with `run\_realtime\_logic` handling sticky realtime acquisition and `run\_campaigns\_logic` handling campaign windows. It also manages subordinate processes, including WebRTC publication, through `ManagedProc`, stale-process cleanup helpers, and explicit stop semantics. The implementation therefore combines logical state management with concrete process-lifecycle discipline.

3.7 Campaign Scheduling and Deferred Execution

Scope. Campaign scheduling is the mechanism by which time-bounded acquisition plans received from the API are translated into executable local behavior.

Technical approach. The design externalizes periodic execution to the operating system's scheduler instead of implementing its own always-awake timing loop for campaign shots. This is methodologically effective for embedded Linux systems because it reduces application complexity while leveraging a mature system scheduler.

Code realization. The scheduler is implemented by `CronSchedulerCampaign` in `functions.py`. It synchronizes API campaign metadata, chooses the highest-priority valid campaign, mirrors its key RF parameters into `ShmStore`, and writes a crontab entry pointing to `campaign\_runner`.

py. The runner then performs a single acquisition cycle, uploads the result, and applies local storage policies when network delivery fails.

3.8 Inter-process Communication and Shared State

Scope. This section covers the mechanisms by which the separate executables cooperate without being collapsed into a monolithic process.

Technical approach. Two complementary integration mechanisms are used. Request-response control and bulk spectral results use ZeroMQ over an IPC endpoint, which is well suited to synchronous command execution between the Python control plane and the C RF engine. Low-frequency persistent node state uses a JSON file in `/dev/shm`, which offers low-latency cross-process visibility while avoiding unnecessary writes to flash storage.

Code realization. The ZMQ path is implemented by `ZmqPairController` in `utils/request\_util.py` and by `rf/libs/zmq\_util.[ch]` in the native layer. The controller enforces strict request-reply semantics and recreates the socket after timeouts to discard late replies. Shared state is implemented by `ShmStore` in `utils/io\_util.py`, which uses file locking and atomic update patterns. The GPS-LTE service uses analogous C helpers in `gps-lte/libs/utils.c` so that both language domains converge on the same persistence model.

3.9 Data Integrity, Retry, and Offline Survivability

Scope. This section examines the mechanisms that preserve measurement artifacts during transient API or network failures.

Technical approach. The design follows an eventually-deliverable telemetry model. If a measurement cannot be uploaded immediately, it should either remain locally recoverable or be discarded only under explicit resource constraints. The system therefore combines atomic file persistence, bounded queueing, and a dedicated retry worker.

Code realization. Atomic persistence is implemented by `atomic\_write\_bytes` in `utils/io\_util.py`. Campaign samples are stored to `Queue/` when uploads fail and to `Historic/` when uploads succeed and disk conditions permit. The dedicated script `retry\_queue.py` reads queued JSON files oldest-first, distinguishes transient and permanent HTTP failures, deletes corrupt payloads, and preserves ordering by stopping on the first unsent file. This is a clear and deliberate resilience policy rather than incidental error handling.

3.10 Observability and Operational Telemetry

Scope. The node requires a self-reporting capability so that remote operators can assess not only measurement outputs but also sensor health.

Technical approach. Observability is treated as a first-class operational requirement. The node therefore reports both static identity metadata and dynamic health metrics such as CPU load, memory usage, disk use, temperature, ping latency, calibration timestamp, and last NTP

synchronization. This is methodologically important because unattended field nodes cannot be managed exclusively through payload success rates.

Code realization. The reporting path is implemented by `status.py` and `utils/status\_util.py`. `StatusDevice` reads Linux virtual files such as `/proc/stat` and `/proc/meminfo`, extracts recent log fragments, and assembles a normalized status structure via `StatusPost`. The final payload is transmitted through `RequestClient`. Logging itself is hardened by `cfg.py`, which defines `AtomicRotator` and environment-sensitive logging filters.

3.11 Audio Streaming and Remote Listening

Scope. The system includes an optional audio-monitoring path for demodulated signals, distinct from the spectral data path.

Technical approach. The design uses a split streaming path: native code performs demodulation and Opus-oriented audio preparation, while a dedicated Python process bridges the encoded audio into a WebRTC session. This separation is methodologically strong because WebRTC negotiation and GStreamer lifecycle handling are complex control-plane concerns that should not burden the RF executable.

Code realization. In the native layer, `audio\_thread\_fn` in `rf/rf.c` drains IQ samples from a dedicated audio ring buffer, optionally filters them, demodulates AM or FM, and hands frames to the TCP/Opus transmission support libraries `rf/libs/opus\_tx.h` and `rf/libs/net\_audio\_retry.h`. In Python, `server\_webrtc.py` accepts the TCP stream, builds a GStreamer pipeline around `webrtcbin`, and maintains signaling with the remote server. The orchestrator starts and stops this process according to demodulation mode.

Chapter 4

Design Rationale and Quality Attributes

4.1 Determinism and Timing Discipline

Scope. This section evaluates how the repository preserves timing predictability in the portions of the node that interact directly with RF hardware and streaming buffers.

Technical approach. Determinism is promoted by reducing callback work, using preallocated or reusable buffers, separating audio and PSD paths, and keeping high-level orchestration out of the RF process. Timing discipline in SDR systems is less about hard real-time certification than about avoiding systematic sample loss, reconfiguration races, and unbounded latency spikes.

Code realization. Evidence includes the minimal `rx\_callback`, ring-buffer usage in `rf/libs/ring\_buffer.c`, RF processing workspaces in `rf/rf.c`, and the request cooldown mechanism propagated from `ServerRealtimeConfig` through `parse\_config\_rf` into the RF engine's runtime state.

4.2 Fault Containment and Recoverability

Scope. The system must continue operating or degrade safely when hardware, network, or subordinate processes fail.

Technical approach. Fault containment is achieved through process separation, explicit health checks, bounded retries, and staged recovery. The methodological aim is not absolute fault elimination, but controlled failure domains and recoverable degradation.

Code realization. The RF engine includes health checks such as `ensure\_hackrf\_session\_is\_healthy` and `recover\_hackrf`. The orchestrator uses stale-process cleanup and managed subprocess wrappers. Upload failures are handled by queue persistence and retry workers. The GPS-LTE service contains PPP reconnection logic. These mechanisms collectively show that operational failure was anticipated as a normal design condition.

4.3 Maintainability and Extension Potential

Scope. This section addresses how easily the codebase can be modified for new campaigns, additional RF modes, or deployment changes.

Technical approach. Maintainability in embedded monitoring systems depends on module boundaries more than on language uniformity. The repository favors explicit modules, small service entry points, configuration objects, and domain-specific helper libraries. This supports local reasoning and selective replacement.

Code realization. Examples include the typed Python configuration dataclasses, the dedicated DSP libraries under `rf/libs/`, the explicit GPS/LTE helper libraries, and the build-time separation between standard and standalone modes in `CMakeLists.txt` and `build.sh`. A future maintainer can replace a telemetry endpoint, add a new demodulation branch, or alter campaign scheduling with limited impact on unrelated layers.

4.4 Resource Stewardship in an Embedded Context

Scope. The node is intended for resource-constrained deployment, so persistence, logging, and storage policies must avoid unnecessary wear or uncontrolled growth.

Technical approach. The repository consistently adopts flash-aware operational patterns: RAM-backed shared state, atomic file replacement, rotating logs, bounded retry queues, and selective historical retention. These choices align with embedded reliability practice, where write amplification and partial-file corruption are recurring failure sources.

Code realization. The strongest evidence is the use of `/dev/shm/persistent.json`, the `AtomicRotator` log manager in `cfg.py`, the queue and historic directories managed by `campaign\_runner.py`, and the immediate `fsync`-based writes in `utils/io\_util.py`.

Chapter 5

Build, Deployment, and Documentation Method

5.1 Compilation Strategy

Scope. This section documents how the node is built and how the build structure reflects architectural intent.

Technical approach. The build method distinguishes target deployment from host-side development. This is a critical methodological feature because hardware-bound projects often stall if local development requires the full peripheral stack.

Code realization. The top-level `CMakeLists.txt` defines a standard build, which produces both `rf\_app` and `ltegps\_app`, and a standalone build activated with `BUILD\_STANDALONE`, which removes the GPIO dependency and compiles only the RF application. `build.sh` exposes this distinction through its default mode and its `-dev` mode.

5.2 Deployment and Service Registration

Scope. The deployment method must provision dependencies, artifacts, shared state, and operating system services in a reproducible way.

Technical approach. The repository uses a scripted installation workflow that approximates infrastructure as code. Although implemented as shell rather than a configuration-management framework, the script is methodologically explicit about service shutdown, dependency installation, compilation, permission repair, shared-memory initialization, and systemd enablement.

Code realization. These responsibilities are captured in `install.sh`. The script installs system packages, optionally compiles `libgpiod` and `kalibrate-hackrf`, creates the Python virtual environment, compiles native binaries, initializes `/dev/shm/persistent.json`, registers systemd services through `init\_sys.py` and `daemons/*.service`, and reboots the system at the end.

5.3 Documentation Pipeline

Scope. A mature engineering repository should document not only runtime behavior but also how documentation artifacts are generated and refreshed.

Technical approach. The repository uses a mixed documentation method aligned with its mixed-language implementation: Doxygen for C-level structural documentation and Sphinx for higher-level rendered documentation. This is methodologically consistent with the source organization and avoids forcing one toolchain to understand both language domains equally well.

Code realization. The automation scripts `document.sh` and `build\_docs.sh` create the documentation environment, run Doxygen, and build Sphinx outputs. The files `docs/python\_modules.rst` and `docs/c\_modules.rst` already define the intended documentation entry points for the Python and C layers respectively.

Chapter 6

Verification Status and Methodological Gaps

6.1 Current Verification Evidence

Scope. This section documents what forms of validation are currently present in the repository.

Technical approach. Verification evidence is evaluated conservatively. The existence of scripts or examples is not treated as equivalent to a comprehensive automated test suite; rather, each artifact is classified according to the level of assurance it can reasonably provide.

Code realization. The repository contains build verification assets, documentation builds, benchmarking tools, and operational test scripts. Examples include `test/test\_benchmarking.py`, `test/tester.py`, `test/hackrf-max-5min.py`, and the benchmarking support under `utils/benchmarking.py` and `examples/benchmark\_rf\_example.py`. These artifacts provide useful performance and manual-operational evidence, but they do not yet constitute broad unit or integration-test coverage across the full control plane and native DSP path.

6.2 Limitations Identified During Inspection

Scope. A methodological report should record not only strengths but also explicit limitations that affect future assurance work.

Technical approach. The principal limitation is that verification is weighted toward operational scripts and benchmarking rather than systematic automated assertions over module contracts, failure injection, and cross-process integration scenarios. This does not invalidate the architecture, but it constrains the level of confidence that can be claimed from repository-local evidence alone.

Code realization. The most visible gaps are the absence of a broad automated test harness for the RF request contract, shared-memory concurrency semantics, and orchestrator state transitions under exceptional conditions. Additional value would come from deterministic tests around `ServerRealtimeConfig`, `ShmStore`, retry semantics in `retry\_queue.py`, and parser boundary handling in `rf/libs/parser.c`.

6.3 Recommended Verification Priorities

Scope. This subsection records technically grounded next steps that follow directly from the inspected architecture.

Technical approach. Recommended actions are prioritized by architectural leverage: protect interfaces first, then validate fault paths, then measure performance regressions. Such sequencing yields faster quality improvement than beginning with broad end-to-end hardware tests.

Code realization.

1. add pure-software tests for configuration validation and Nyquist clamping;
2. add concurrency-oriented tests for `ShmStore` update semantics;
3. add orchestrator tests that simulate API responses and verify correct state transitions;
4. add contract tests for retry queue behavior under 2xx, 4xx, 5xx, and timeout conditions;
5. preserve the existing benchmarking scripts as regression monitors for throughput and thermal behavior on target hardware.

Chapter 7

Traceability Matrix

7.1 Requirement-to-Module Allocation

Scope. The following matrix maps architectural responsibilities to the modules that currently realize them. It is intended to support maintenance, review, and change-impact analysis.

Technical approach. Traceability is organized by engineering concern rather than by folder alone. This improves its usefulness during audits because reviewers typically reason from required behavior toward source artifacts, not in the reverse direction.

Code realization.

Engineering concern	Primary modules	Methodological contribution
RF acquisition and PSD	<code>rf/rf.c</code> , <code>rf/libs/psd.c</code> , <code>rf/libs/ring_buffer.c</code>	Implements the deterministic data path, workspace reuse, ring-buffer decoupling, and PSD generation.
Runtime request validation	<code>utils/request_util.py</code> , <code>rf/libs/parser.c</code>	Applies typed validation, defaults, and Nyquist-safe clamping before hardware reconfiguration.
Frequency calibration	<code>orchestrator.py</code> , <code>rf/rf.c</code> , <code>rf/libs/sdr_HAL.c</code>	Estimates and persists <code>ppm_error</code> , then applies corrected tuning frequencies during operation.
GPS and LTE telemetry	<code>gps-lte/gps-lte.c</code> , <code>gps-lte/libs/bacn_GPS.c</code> , <code>gps-lte/libs/bacn_LTE.c</code>	Maintains geolocation and network connectivity in an isolated service domain.
GPIO and antenna routing	<code>common/bacn_gpio.c</code> , <code>common/bacn_gpio.h</code>	Centralizes board-level control signals and preserves portability through build-time exclusion.
Orchestration and system modes	<code>orchestrator.py</code> , <code>functions.py</code>	Defines the global state machine, real-time loop, calibration trigger path, and campaign supervision.
Campaign execution	<code>functions.py</code> , <code>campaign_runner.py</code>	Translates server campaign windows into scheduled local acquisitions and controlled uploads.

Engineering concern	Primary modules	Methodological contribution
IPC and shared persistence	utils/request_util.py, rf/libs/zmq_util.h, utils/io_util.py	Provides synchronous command exchange and low-wear cross-process state sharing.
Offline survivability	campaign_runner.py, retry_queue.py, utils/io_util.py	Ensures measurements remain recoverable across network outages and partial API failures.
Status and observability	status.py, utils/status_util.py, cfg.py	Collects health metrics, log context, timing metadata, and environment-sensitive diagnostics.
Audio publication	rf/rf.c, rf/libs/opus_tx.h, server_webrtc.py	Supports remote listening while isolating signaling complexity from the RF executable.
Build and deployment	CMakeLists.txt, build.sh, install.sh	Encodes reproducible compilation and target provisioning with explicit dev/target separation.

Chapter 8

Conclusion

8.1 Synthesis

Scope. This concluding section consolidates the methodological interpretation of the repository as a whole.

Technical approach. The node should be understood as a deliberately layered sensing platform rather than as a single application. Its architecture reflects an embedded-systems design method in which hardware- proximate, timing-sensitive, and buffer-heavy responsibilities remain in C, while coordination, policy, networking, and operator-facing workflows remain in Python. The integration strategy relies on simple, inspectable mechanisms: synchronous IPC over ZeroMQ and RAM-backed JSON persistence.

Code realization. The repository expresses this method consistently across its implementation and tooling: the RF engine, GPS-LTE service, orchestration loop, campaign runner, retry worker, status reporter, and documentation pipeline are distinct but interoperable artifacts. The strongest architectural qualities evidenced by the code are separation of concerns, resilience to intermittent failures, flash-aware persistence discipline, and a clear path for incremental enhancement. The main outstanding engineering need is broader automated verification around contract boundaries and failure scenarios.

Appendix A

Key Source Inventory

Path	Primary role
rf/rf.c	Main RF executable, acquisition loop, calibration, audio thread, JSON reply generation.
rf/libs/parser.c	JSON parsing, defaults, and boundary enforcement for RF requests.
rf/libs/psd.c	Spectral estimation algorithms and parameter derivation.
rf/libs/sdr_HAL.c	HackRF hardware configuration with PPM-aware tuning.
gps-lte/gps-lte.c	GPS acquisition, LTE connectivity management, and coordinate reporting.
common/bacn_gpio.c	GPIO control for modem power/reset and antenna selection.
orchestrator.py	Global supervisor for realtime and campaign modes.
functions.py	Shared orchestration helpers, state model, scheduler, acquisition wrapper.
campaign_runner.py	One-shot campaign acquisition and upload worker.
retry_queue.py	Deferred upload recovery worker.
status.py	Status payload generation and publication entry point.
server_webrtc.py	WebRTC/GStreamer bridge for remote audio monitoring.
utils/io_util.py	Atomic writes, shared-memory persistence, timers.
utils/request_util.py	HTTP client, ZMQ controller, configuration dataclasses.
utils/status_util.py	Host-health telemetry collection and shaping.
cfg.py	Environment loading, paths, identity helpers, and logging infrastructure.
CMakeLists.txt, build.sh, install.sh	Build and deployment baseline.